

Informe de Práctica de Laboratorio Especializado

Path Traversal — Traversal sequences stripped with superfluous URL-decode

PortSwigger Web Security Academy — Nivel Practitioner

Contenido

1. Introducción

Este informe documenta el laboratorio “File path traversal, traversal sequences stripped with superfluous URL-decode” de PortSwigger Web Security Academy (nivel Practitioner). En este caso, la aplicación primero bloquea la entrada que contiene secuencias de traversal explícitas y, después de esa validación, realiza una decodificación URL adicional (superflua) sobre el valor recibido antes de utilizarlo.

Esta segunda decodificación, realizada después del filtro, permite enviar una secuencia de traversal codificada en URL que pasa el filtro inicial sin ser detectada, para luego decodificarse y convertirse en una secuencia “../” funcional justo antes de ser usada por la aplicación.

El ejercicio se realizó en el entorno de PortSwigger, empleando Burp Suite Community Edition como proxy de interceptación.

2. Objetivos

Objetivo general

Demostrar la capacidad de evadir un filtro de path traversal aprovechando una decodificación URL superflua aplicada después de dicho filtro, mediante el uso de codificación URL sobre el payload.

Objetivos específicos

- Configurar Burp Suite Community Edition como proxy de interceptación.
- Confirmar que el envío directo de secuencias “../” es bloqueado por la aplicación.
- Comprender el concepto de decodificación URL superflua y su riesgo asociado.
- Construir un payload codificado en URL que evada el filtro y se decodifique en una secuencia de traversal válida.
- Acceder al contenido del archivo /etc/passwd.
- Proponer medidas de mitigación adecuadas.

3. Preparación del laboratorio

El laboratorio “File path traversal, traversal sequences stripped with superfluous URL-decode” es de nivel Practitioner. La aplicación bloquea la entrada que contiene secuencias de path traversal, pero luego realiza una decodificación URL adicional del valor recibido antes de usarlo. El objetivo es recuperar el contenido de `/etc/passwd`.

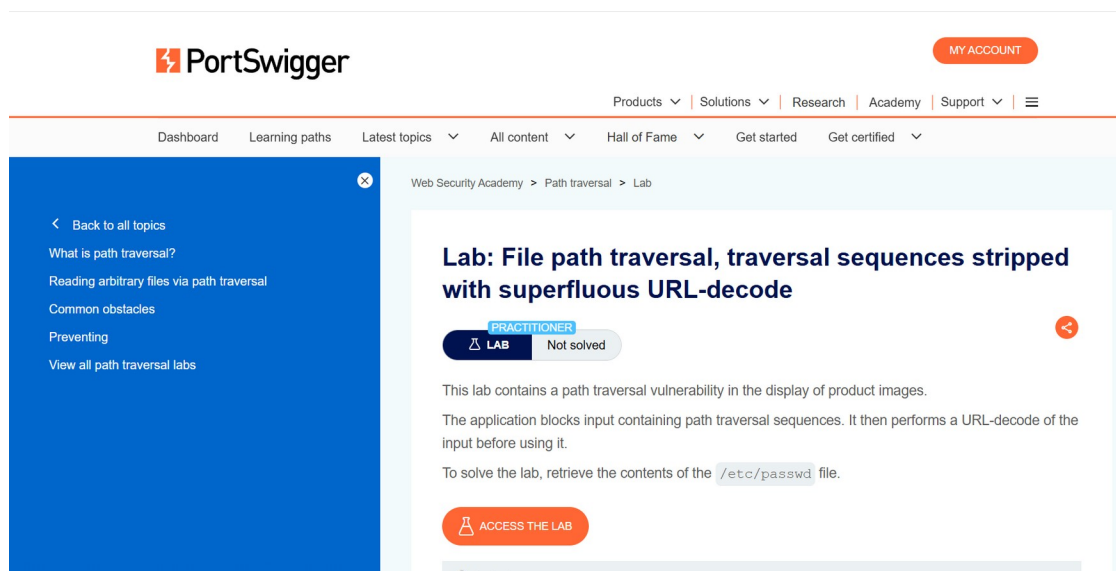


Figura 1. Descripción del laboratorio “Traversal sequences stripped with superfluous URL-decode” en PortSwigger.

Concepto teórico: decodificación URL superflua. Cuando una aplicación web recibe una petición HTTP, el servidor o el framework normalmente ya decodifica una vez los caracteres codificados en URL (por ejemplo, `%2e%2e%2f`) antes de que el código de la aplicación reciba el valor. Si, además de esa decodificación automática, el propio código de la aplicación realiza una segunda decodificación URL manual sobre el valor —por ejemplo, para “asegurarse” de que no queden caracteres codificados—, se introduce una decodificación superflua (innecesaria). Esta segunda decodificación puede ocurrir después de que el filtro de seguridad ya validó la entrada, abriendo una ventana de evasión.

4. Fase 1: Interceptación de la petición de imagen

Con el proxy activo, se navegó por el catálogo de “We Like To Shop”. En el historial de peticiones se identificó el patrón habitual de solicitudes GET para cargar cada imagen de producto.

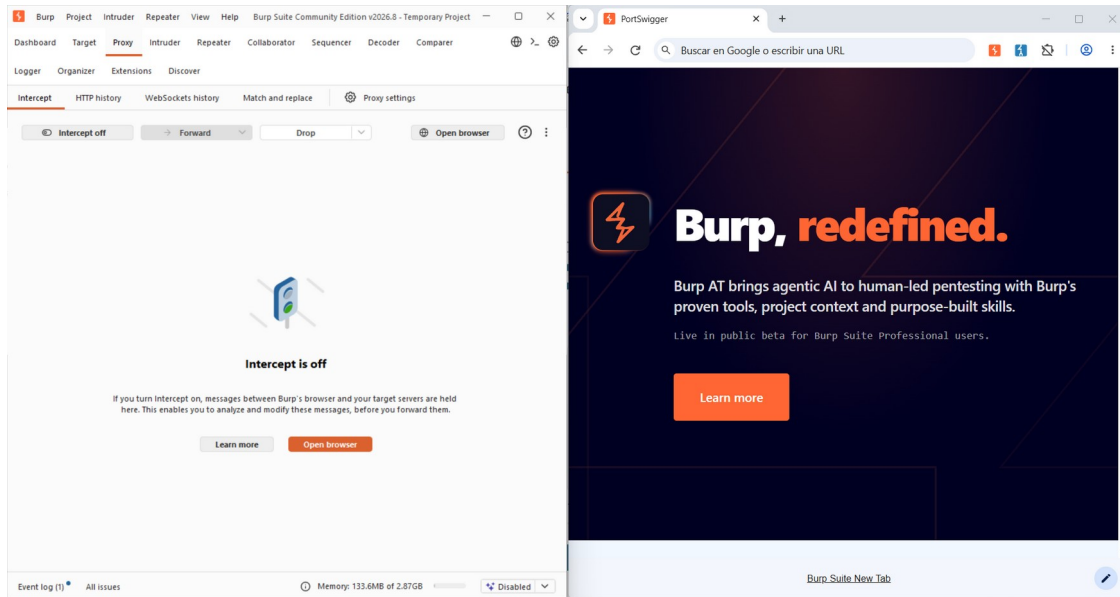


Figura 2. Página de la tienda “We Like To Shop” e historial de peticiones GET /image?filename=N.jpg en Burp Proxy.

Se seleccionó una de estas peticiones desde el historial de HTTP history para enviarla a Repeater.

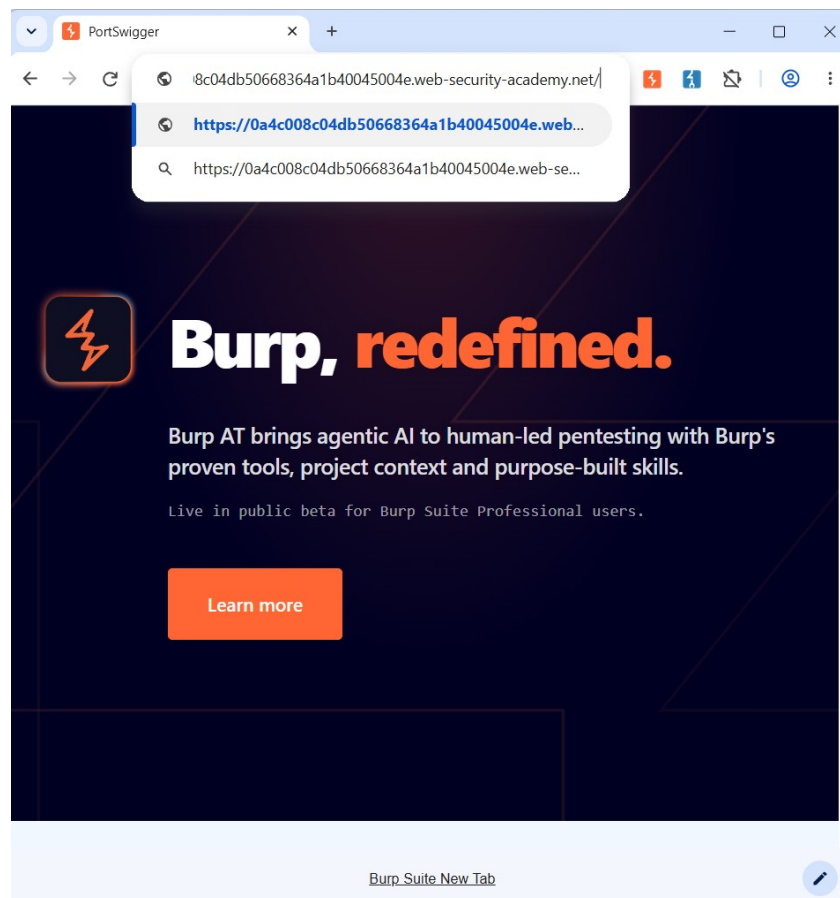


Figura 3. Historial de peticiones GET /image?filename=N.jpg, mostrando código de estado, longitud y tipo MIME de cada respuesta.

5. Fase 2: Envío a Repeater y confirmación del filtro

La petición seleccionada (filename=43.jpg) se envió a Repeater mediante “Send to Repeater” para poder modificar el parámetro de forma controlada.

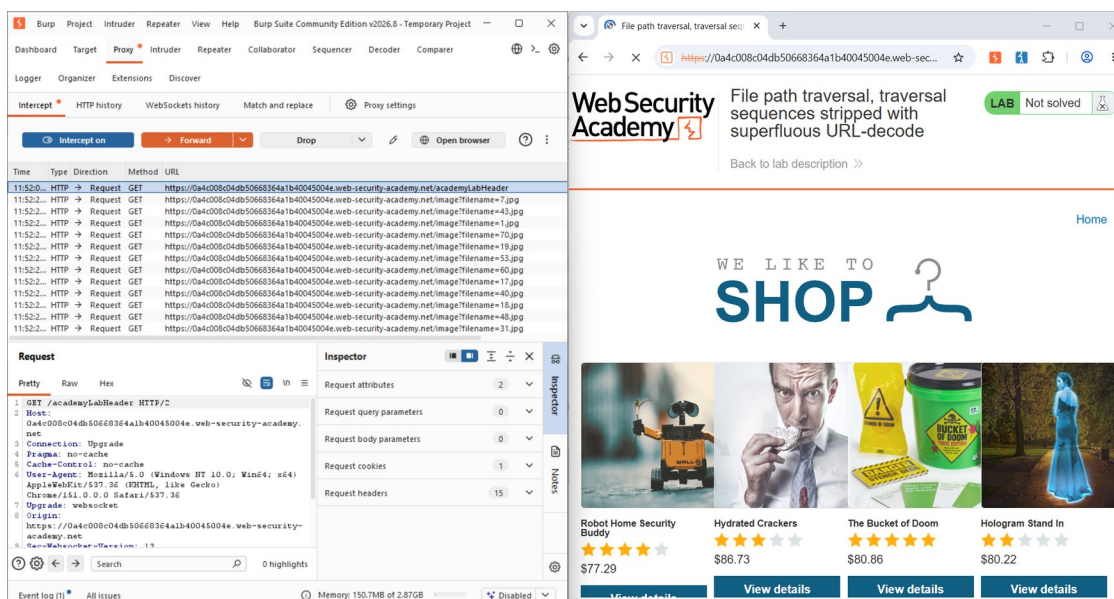


Figura 4. Envío de la petición seleccionada (filename=43.jpg) a Burp Repeater mediante el menú contextual.

Dentro de Repeater se confirmó la petición original antes de modificarla.

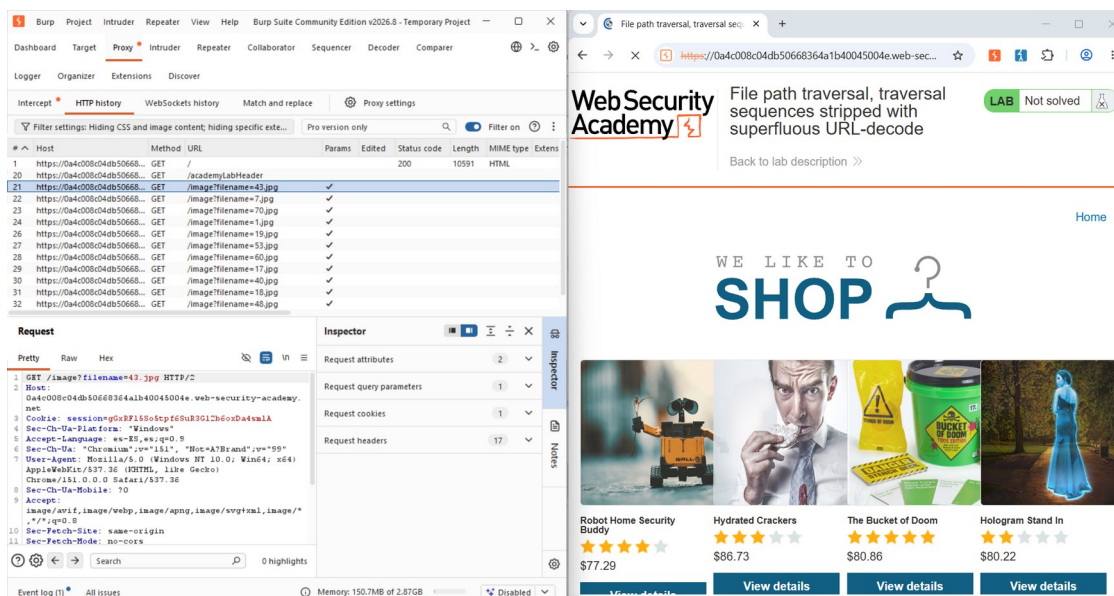


Figura 5. Petición original GET /image?filename=43.jpg visible en Repeater.

Un intento directo con el payload clásico ../../etc/passwd sería bloqueado por el filtro inicial de la aplicación, que detecta la presencia de la secuencia literal “../” en la entrada recibida.

6. Fase 3: Construcción del payload con doble codificación URL

Para evadir el filtro, se codificó en URL cada carácter de la secuencia de traversal, de modo que el filtro inicial no reconociera el patrón “../” de forma literal, confiando en que una decodificación posterior (superflua) de la propia aplicación revelaría la secuencia real justo antes de usarla:

```
GET /image?filename=..%252f..%252f..%252fetc/passwd HTTP/2
```

Aquí, %252f es la codificación URL del carácter %2f (que a su vez es la codificación URL de la barra “/”). Es decir, el punto se deja literal, pero la barra se codifica dos veces.

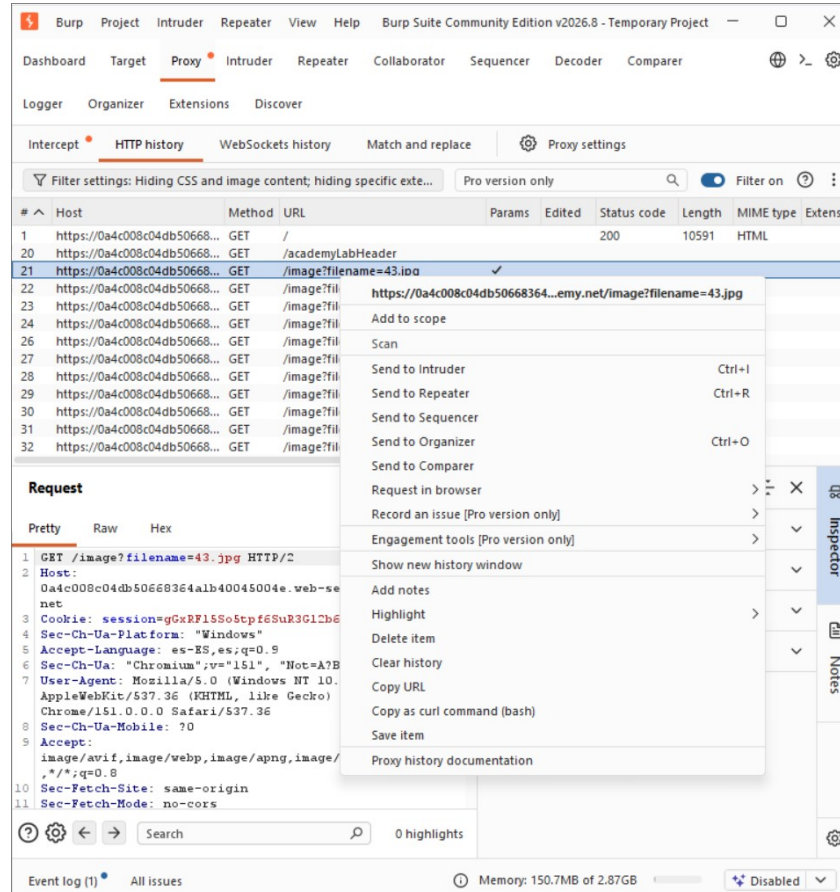


Figura 6. Petición modificada en Repeater con el payload doblemente codificado `..%252f..%252f..%252fetc/passwd`.

7. Fase 4: Explotación y confirmación

Al enviar la petición, el servidor respondió con un código HTTP 200 OK y devolvió el contenido íntegro del archivo `/etc/passwd`, confirmando la explotación exitosa.

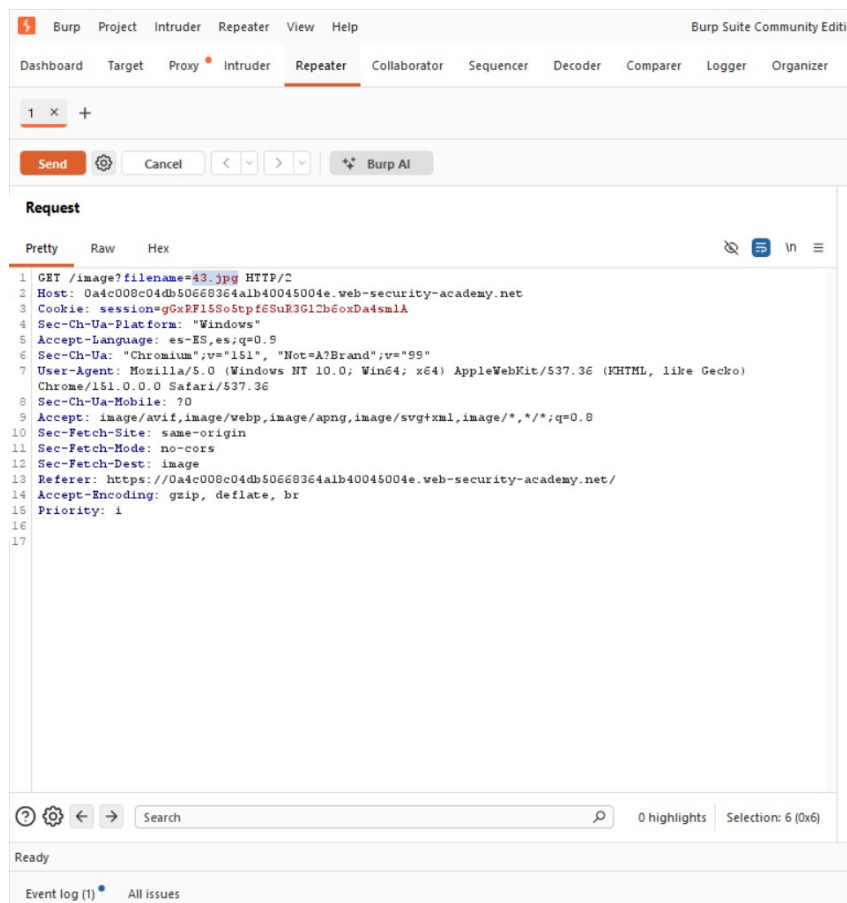


Figura 7. Respuesta del servidor con el contenido completo de `/etc/passwd` tras enviar el payload doblemente codificado.

7.1 Explicación del payload

El funcionamiento del payload puede desglosarse en tres etapas de procesamiento:

- Primera decodificación (automática, del servidor/framework): al recibir la petición, el servidor decodifica una vez los caracteres URL-encoded. `%252f` se decodifica a `%2f` (todavía codificado una vez).
- Filtro de seguridad: la aplicación examina el valor resultante (`..%2f..%2f..%2fetc/passwd`) buscando la secuencia literal `"/"`. Como en este punto la barra sigue codificada como `%2f` y no como el carácter `"/"`, el filtro no detecta ninguna secuencia de traversal y deja pasar la entrada.
- Segunda decodificación (superflua, de la aplicación): la aplicación decodifica nuevamente el valor ya validado, transformando `%2f` en `"/"`. En este punto, `..%2f` se convierte en `"/"`, reconstruyendo la secuencia de traversal justo después de haber pasado el filtro.

El resultado final que llega a la operación de acceso a archivos es ../../etc/passwd, idéntico al payload clásico, pero que logró atravesar el filtro gracias al desfase entre el momento de la validación y el momento de la decodificación.

Concepto teórico: desincronización entre validación y decodificación. Este tipo de vulnerabilidad pertenece a una clase más amplia de fallos conocidos como problemas de “parsing diferencial” o desincronización de transformaciones: cuando dos componentes de un sistema (aquí, el filtro y la lógica de decodificación) interpretan o transforman la misma entrada en momentos distintos y de forma distinta, un atacante puede construir una entrada que sea inofensiva para el primer componente pero maliciosa una vez procesada por el segundo. La regla general de mitigación es aplicar cualquier decodificación antes de la validación, nunca después, y evitar decodificaciones redundantes.

7.2 Resolución del laboratorio

Al confirmar la exposición de /etc/passwd, PortSwigger marcó automáticamente el laboratorio como resuelto.

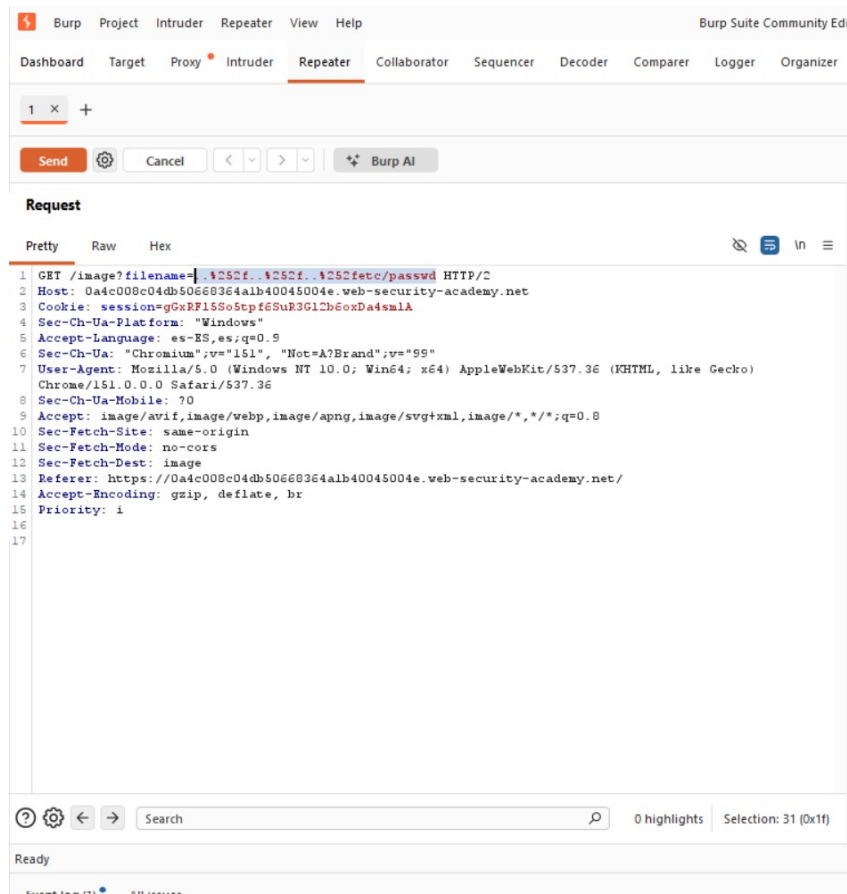


Figura 8. Confirmación de PortSwigger: “Congratulations, you solved the lab!”

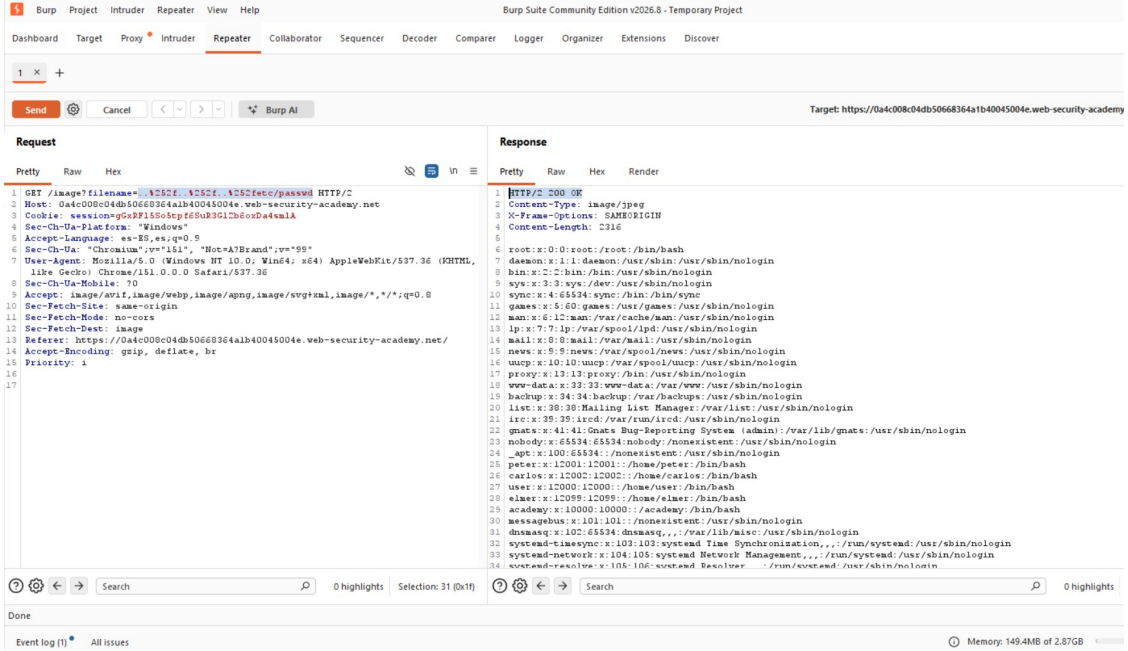


Figura 9. Estado final del laboratorio marcado como “Solved”.

8. Mitigación recomendada

Para prevenir esta clase de bypass basado en decodificación superflua, se recomienda:

- Realizar todas las decodificaciones necesarias sobre la entrada antes de ejecutar cualquier validación de seguridad, nunca después.
- Eliminar decodificaciones redundantes o innecesarias en el código de la aplicación; decodificar el valor una única vez y en el punto correcto del flujo de procesamiento.
- Validar la ruta resultante final (después de cualquier decodificación) contra su forma canónica, verificando que permanezca dentro del directorio base permitido.
- Sustituir el parámetro filename por un identificador indirecto validado contra una lista de archivos permitidos.
- Registrar y auditar el pipeline de procesamiento de entradas para detectar puntos donde se realizan transformaciones repetidas o fuera de orden.

9. Conclusiones

Este laboratorio mostró cómo una decodificación URL adicional, aparentemente inofensiva, puede anular por completo un filtro de seguridad previamente aplicado si ambos procesos no están sincronizados. La lección principal es que el orden de las operaciones en el procesamiento de una entrada es tan importante como las propias operaciones: decodificar después de validar puede convertir una entrada previamente segura en un vector de ataque funcional.

El ejercicio también reforzó la utilidad de Burp Repeater para experimentar con distintos niveles de codificación de un mismo payload, permitiendo observar de forma directa cómo el servidor interpreta cada variante.